Name: Parthiv Patel

GitHub Repository link: https://github.com/Parthivp2205/CPSC-8430

CPSC 8430 Deep Learning

Homework 3

Introduction

This project focuses on developing a complete question answering (QA) system using the BERT Base Uncased model from Hugging Face Transformers. The main goal was to train and evaluate a model capable of finding the most accurate answer span from spoken passages in the Spoken-SQuAD dataset. This dataset contains automatically transcribed speech data, making it a realistic test of how QA models perform under noisy conditions.

The system covers every step of the QA pipeline from loading and preparing the data, to fine-tuning a pretrained transformer model and finally evaluating performance across clean and noisy datasets. By following this end-to-end approach, the project aims to show how transformer models can adapt to speech-based text data while maintaining strong accuracy.

Setup and Preprocessing

The code first checks the hardware setup, confirming that CUDA and GPU acceleration are available. PyTorch version 2.5.1 (CUDA 12.1) was detected, and mixed precision (bf16) was automatically used to make training faster. The environment and random seed were fixed to keep results consistent.

Data Loading and Structure

The data comes from the Spoken-SQuAD dataset, a spoken version of the popular SQuAD reading comprehension benchmark. It contains passages, questions, and corresponding answers with their start positions. The code loads both training and testing JSON files and flattens the structure so that every question-answer pair is stored as a single example containing the context, question, and answers.

Tokenization and Feature Preparation

A fast BERT tokenizer encodes both the question and its context together so that the model can learn relationships between them. Because some passages are longer than the allowed maximum length (384 tokens), the script uses a sliding window technique with a stride of 128 tokens. This means each passage is split into slightly overlapping pieces, so answers near the edge of one window are not missed.

For training, each tokenized example includes the position of the start and end of the answer. If the true answer does not appear in a window, both start and end positions are set to the [CLS] token to signal that the span is not present there. The model uses this structure to learn which tokens in a passage correspond to an answer.

Key hyperparameters:

Batch size: 8 (with gradient accumulation of 2)

Learning rate: 3e-5

Epochs: 3

Warmup ratio: 0.1

Weight decay: 0.01

Random seed: 42

This setup ensures the model trains efficiently while maintaining stable optimization.

Training

The model used in this project is BERT Base Uncased, chosen for its strong performance on text understanding tasks. It was fine-tuned using Hugging Face's Trainer API, which handles batching, optimization, and logging automatically.

During training, the model learned to predict the correct start and end positions of answers based on the question and context. It used the AdamW optimizer and a linear learning rate schedule with warm-up, which gradually increases the learning rate at the beginning to make learning smoother. The process ran for three epochs, and gradient accumulation allowed effective training even with a small batch size.

The loss decreased steadily from around 5.2 to below 1.0, showing that the model was learning effectively. After three epochs, the final training loss was about 1.23, confirming good convergence. The model ran on a GPU and reached around 9.2 training steps per second, finishing training in roughly 12–13 minutes.

The fine-tuned model and tokenizer were saved to an output folder (./spoken_squad_bert_base) for later evaluation and reuse.

Evaluation and Fine-Tuning

After training, the model was evaluated on three different test sets:

1. Clean test set (no noise)
2. WER44 test set – with 44% word error rate
3. WER54 test set – with 54% word error rate

The evaluation follows the same tokenization logic as training, but without labels. The model predicts start and end token logits, and a post-processing step converts these predictions into readable text answers. This process chooses the highest-scoring answer span while ignoring tokens that belong to the question or special symbols.

Two main metrics were used:

Exact Match (EM): how often the model's predicted answer exactly matches the true answer.

F1 Score: how much overlap exists between predicted and true answers, rewarding partial correctness.

The model achieved the following results:

Dataset	**Exact Match (EM)**	**F1 Score**
Dev (Clean)	63.80	74.11
Test (Clean)	63.80	74.11
Test(WER44)	40.74	55.75
Test(WER54)	28.74	42.40

These numbers show that the model performs well on clean text, while performance naturally drops as noise increases. This confirms that the system behaves as expected for speech-based QA, where recognition errors make exact span matching harder.

The evaluation script also saves predictions in JSON files for each dataset, which can be used for manual inspection or further analysis.

Result

Overall, the fine-tuned BERT model demonstrates strong capability in answering questions from spoken text. On clean transcripts, it achieves around 74% F1, which is a solid score for a baseline model. The gradual drop in performance on noisy datasets shows the direct effect of word errors from speech recognition systems.

Despite the drop, the model still manages to extract relevant parts of the text even under high noise, showing that BERT's contextual understanding is quite robust. The clean-to-noisy trend (Clean → WER44 → WER54) aligns well with what is seen in published Spoken-SQuAD research, confirming that the implementation works correctly.

A few things helped improve performance:

- Using overlapping context windows prevented the model from missing answers split across boundaries.
- Mixed-precision training (bf16) made the training process faster and memory-efficient.
- Stable hyperparameter settings and warmup helped avoid large loss spikes.

Possible Improvements

For further improvement, the model could be retrained with:

- BERT-large or RoBERTa variants for higher accuracy.
- Better learning rate schedules, such as cosine decay.
- Noise-aware fine-tuning, where synthetic errors are added during training to make the model more robust.

Conclusion

This project fine-tuned a BERT model to answer questions using the Spoken-SQuAD dataset. It included every main step, getting the data ready, turning text into tokens, training the model, and testing the results. The model gave good results on clean text and still worked well even when the data had noise from speech recognition. Overall, the project built a clear and working example of how BERT can be used for spoken question answering, showing that with the right setup, even a basic model can handle noisy text fairly well